Backpropagation with Negative ReLU Inputs

One iteration: manual gradients and PyTorch verification

Source and scope. The supplied `[HW_Code2]` notebook is titled *DNN Back Propagation with Negative Values at ReLU*. We calculate gradients at its initial weights using one input–target pair and one backward pass, before any weight update. Vectors are row vectors; $(W_\ell)_{ij}$ connects input i to output j. Both layers have no bias.

1. Initial values and forward pass

$$X = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad T = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 2 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} -1 & 2 \\ 1 & 2 \end{bmatrix}.$$

The hidden-layer preactivation and activation are

$$\begin{aligned} S_2 = XW_1 &= \begin{bmatrix} 1(2)+2(0) & 1(1)+2(-1) \end{bmatrix} = \begin{bmatrix} 2 & -1 \end{bmatrix}, \\ Y_2 = \mathrm{ReLU}(S_2) &= \begin{bmatrix} 2 & 0 \end{bmatrix}. \end{aligned}$$

The output-layer preactivation and activation are

$$\begin{aligned} S_3 = Y_2W_2 &= \begin{bmatrix} 2(-1)+0(1) & 2(2)+0(2) \end{bmatrix} = \begin{bmatrix} -2 & 4 \end{bmatrix}, \\ Y_3 = \mathrm{ReLU}(S_3) &= \begin{bmatrix} 0 & 4 \end{bmatrix}. \end{aligned}$$

For $s \neq 0$, $\mathrm{ReLU}'(s) = 1$ when $s > 0$ and 0 when $s < 0$. There are no zero preactivations here. The derivative masks are

$$\mathrm{ReLU}'(S_2) = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad \mathrm{ReLU}'(S_3) = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

2. Loss and output error signal

The default `nn.MSELoss()` averages over both output elements [2], so

$$L = \frac{(y_{3,1}-t_1)^2 + (y_{3,2}-t_2)^2}{2} = \frac{0^2+4^2}{2} = 8.$$

Consequently,

$$\frac{\partial L}{\partial Y_3} = \frac{2}{2}(Y_3 - T) = \begin{bmatrix} 0 & 4 \end{bmatrix}, \qquad \delta_3 = \frac{\partial L}{\partial S_3} = \begin{bmatrix} 0 & 4 \end{bmatrix} \odot \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 4 \end{bmatrix}.$$

Here $\odot$ denotes elementwise multiplication. The first output ReLU blocks its gradient because its input is negative; in this example its output residual is also zero because $y_{3,1} = t_1 = 0$.

3. Gradient of the second-layer weights

Since $s_{3,j} = \sum_i y_{2,i}(W_2)_{ij}$, the chain rule gives $\partial L/\partial (W_2)_{ij} = y_{2,i}\delta_{3,j}$. Thus

$$\boxed{\nabla_{W_2} L = Y_2^{\mathsf{T}}\delta_3 = \begin{bmatrix} 2\cdot 0 & 2\cdot 4 \\ 0\cdot 0 & 0\cdot 4 \end{bmatrix} = \begin{bmatrix} 0 & 8 \\ 0 & 0 \end{bmatrix}.}$$

The second row is zero because the second hidden activation is zero.

4. Hidden error signal and first-layer gradients

Before applying the hidden ReLU derivative, each hidden activation receives the sum of its contributions to both output errors:

$$\frac{\partial L}{\partial y_{2,i}} = \sum_j \delta_{3,j}(W_2)_{ij}, \qquad \frac{\partial L}{\partial Y_2} = \delta_3 W_2^{\mathsf{T}}.$$

Substituting the values,

$$\delta_3 W_2^{\mathsf{T}} = \begin{bmatrix} 0 & 4 \end{bmatrix} \begin{bmatrix} -1 & 1 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 0(-1)+4(2) & 0(1)+4(2) \end{bmatrix} = \begin{bmatrix} 8 & 8 \end{bmatrix}.$$

The second hidden preactivation is -1, so its ReLU derivative is zero:

$$\delta_2 = \frac{\partial L}{\partial S_2} = (\delta_3 W_2^{\mathsf{T}}) \odot \mathrm{ReLU}'(S_2) = \begin{bmatrix} 8 & 8 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 8 & 0 \end{bmatrix}.$$

Since $s_{2,j} = \sum_i x_i (W_1)_{ij}$,

$$\frac{\partial L}{\partial (W_1)_{ij}} = x_i \delta_{2,j}, \qquad \boxed{\nabla_{W_1} L = X^{\mathsf{T}} \delta_2 = \begin{bmatrix} 1\cdot 8 & 1\cdot 0 \\ 2\cdot 8 & 2\cdot 0 \end{bmatrix} = \begin{bmatrix} 8 & 0 \\ 16 & 0 \end{bmatrix}.}$$

Both weights feeding the second hidden neuron have zero gradient: the negative preactivation blocks the nonzero upstream derivative 8.

5. Verification with the executed notebook

The notebook's `torch.matmul` implementation prints

$$\texttt{W1.grad} = \begin{bmatrix} 8 & 0 \\ 16 & 0 \end{bmatrix}, \qquad \texttt{W2.grad} = \begin{bmatrix} 0 & 8 \\ 0 & 0 \end{bmatrix}, \qquad Y_3 = \begin{bmatrix} 0 & 4 \end{bmatrix}.$$

These exactly match the manual calculations, and the recorded loss is 8.

Accounting for `nn.Linear`. PyTorch computes $xA^{\mathsf{T}} + b$ and stores A with shape (output features, input features) [1]. For an equivalent network, set

$$A_1 = W_1^{\mathsf{T}} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \end{bmatrix}, \qquad A_2 = W_2^{\mathsf{T}} = \begin{bmatrix} -1 & 1 \\ 2 & 2 \end{bmatrix}.$$

Since $(A_\ell)_{ji} = (W_\ell)_{ij}$, its parameter gradients are

$$\nabla_{A_\ell} L = (\nabla_{W_\ell} L)^{\mathsf{T}}, \qquad \nabla_{A_1} L = \begin{bmatrix} 8 & 16 \\ 0 & 0 \end{bmatrix}, \quad \nabla_{A_2} L = \begin{bmatrix} 0 & 0 \\ 8 & 0 \end{bmatrix}.$$

The added verification cell checks the manual matrices against both direct gradients and both transposed `nn.Linear` gradients. All four checks pass with maximum absolute error 0. It also checks both preactivations, both activations, and the loss against the values above.

References

[1] PyTorch documentation, `torch.nn.Linear`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.Linear.html`

[2] PyTorch documentation, `torch.nn.MSELoss`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.MSELoss.html`